

Московский физико-технический институт
Высшая школа программной инженерии

Алгоритмы
3 семестр, 2026/2027 уч. год

Лекция 2

Точный поиск множества паттернов в тексте

Лектор: Кулапин Артур
Автор: [Ivan Gerunov](#)

1. Точный поиск множества паттернов в тексте

В прошлой лекции мы искали в тексте один паттерн. Теперь паттернов много: $P_1, \dots, P_k$, и хочется найти вхождения их всех сразу. Можно, конечно, k раз запустить КМП, но это $O\left(k |T| + \sum_{i=1}^k |P_i|\right)$ — текст мы прочитаем k раз. Цель лекции — прочитать текст **один** раз и за это время найти все вхождения всех паттернов.

Примечание. Эта лекция во многом продолжает первую: Ахо-Корасик — это, по сути, КМП для многих паттернов, и почти каждое рассуждение ниже опирается на префикс-функцию. Особенно пригодится понимание того, почему откат $k = \pi[k]$ перебирает все супрефиксы по убыванию длины и почему КМП работает за линию, хотя откатов подряд бывает много. Если что-то из этого пока не до конца уложилось — стоит сначала ненадолго вернуться к первой лекции, тогда здесь всё сложится гораздо легче.

1.1 Бор

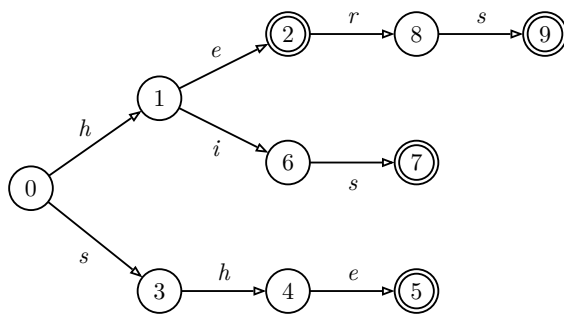
Определение 1 (Бор). Бор — структура данных для хранения набора строк, представляющая собой подвешенное дерево с символами на рёбрах. Строки получают последовательной записью всех символов, хранящихся на рёбрах между корнем бора и терминальной вершиной.

По сути бор — это «словарь с общими префиксами», поэтому его ещё называют «префиксным деревом». Если две строки начинаются одинаково, то общее начало хранится один раз, а в месте, где они расходятся, дерево ветвится.

Определение 2 (Слово вершины). Для вершины u бора через $[u]$ обозначим слово, которое читается на пути от корня до u . Обратно, $\text{vertex}(w)$ — вершина, в которую ведёт путь с надписью w (если такой путь есть).

Примечание: $[u]$ — это не обязательно одна из исходных строк, а может быть просто префиксом какой-то из исходных строк. Исходным строкам соответствуют терминальные вершины.

Пример 1. Бор для набора образцов *he, she, his, hers* (терминальные вершины обведены двойной линией):



Здесь, например, $[4] = \text{«sh»}$, $[8] = \text{«her»}$, а $[9] = \text{«hers»}$. Вершина 8 не терминальная: слово «her» в боре читается, но в наборе его нет.

Алгоритм 1: Операции в боре

1. **Поиск строки S .** Идём от корня по буквам S . Если в какой-то момент нужного ребра нет, или мы дочитали S , но оказались не в терминальной вершине — строки в наборе нет.
2. **Добавление строки S .** Идём от корня по буквам S , пока можем. Как только нужного ребра нет — дописываем остаток S новой веткой. Последнюю вершину помечаем терминальной.
3. **Удаление строки S .** Находим вершину $\text{vertex}(S)$ и снимаем с неё пометку терминальности. Далее удаляем вершины с конца, пока не наткнёмся на терминальную вершину или на вершину, у которой больше одного ребёнка (эти вершины нужны другим строкам). Отдельно: если $\text{vertex}(S)$ — не лист (например, удаляем «he», а в боре есть «hers»), то удалять вершины нельзя — они нужны другим строкам, достаточно снять пометку. А если небольшой избыток памяти допустим ради скорости работы, можно и всегда ограничиваться снятием пометки, вообще не удаляя вершины.

Время работы

Каждая операция делает не больше $|S|$ шагов по рёбрам, так что это $O(|S|)$ переходов. Сколько стоит один переход — зависит от того, как в вершине хранятся рёбра (см. таблицу ниже).

Внутри вершины у нас есть переходы по буквам, но как их хранить? Сравним варианты (здесь dict — набор строк в боре, $|S|$ — длина искомой строки):

Контейнер	Построение	Поиск	Память
массив на $ \Sigma $	$O\left(\sum_{w \in \text{dict}} w \right)$	$O(S)$	$O\left(\Sigma \sum_{w \in \text{dict}} w \right)$
map	$O\left(\log \Sigma \sum_{w \in \text{dict}} w \right)$	$O(S \log \Sigma)$	$O\left(\sum_{w \in \text{dict}} w \right)$
unordered_map	$O\left(\sum_{w \in \text{dict}} w \right)$ в среднем	$O(S)$ в среднем	$O\left(\sum_{w \in \text{dict}} w \right)$
список рёбер	$O\left(\Sigma \sum_{w \in \text{dict}} w \right)$	$O(S \cdot \Sigma)$	$O\left(\sum_{w \in \text{dict}} w \right)$

Массив — самый быстрый, но на каждую вершину тратит $|\Sigma|$ ячеек, даже если ребёнок у неё один. Строго говоря, из-за этого и построение у него стоит $O\left(|\Sigma| \sum_{w \in \text{dict}} |w|\right)$ — каждую новую вершину надо заполнить $|\Sigma|$ пустыми ссылками; в таблице (как и на слайдах) алфавит считается константой. При маленьком алфавите (буквы, цифры, 0/1) это не страшно, и дальше мы будем пользоваться именно им. Список рёбер (каждая вершина хранит номер первого ребра, каждое ребро — номер следующего) экономит память, но поиск ребра в нём — перебор до $|\Sigma|$ вариантов.

В реализации ниже, кроме флага терминальности, в вершине хранится номер паттерна, который в ней заканчивается: при поиске нам нужно знать, *какой именно* паттерн нашёлся. Считаем, что одинаковых паттернов в наборе нет. Вершины и их массив объявлены **protected**: они понадобятся наследнику — автомату Ахо-Корасик.

Чего мы добились

Реализация (C++): бор

```
const int kAlphabetSize = 26;
const int kRoot = 0;
const int kNoNode = -1;

class Trie {
public:
    Trie() {
        nodes_.push_back(Node()); // бор изначально состоит из одного корня
    }

    void add_pattern(const std::string& pattern, int id) {
        int node = kRoot;
        int length = pattern.size();
        for (int index = 0; index < length; ++index) {
            int letter = pattern[index] - 'a';
            if (nodes_[node].children[letter] == kNoNode) { // не смогли пойти —
                nodes_[node].children[letter] = nodes_.size(); // дописываем новую ветку
                nodes_.push_back(Node());
            }
            node = nodes_[node].children[letter];
        }
        nodes_[node].is_terminal = true;
        nodes_[node].pattern_id = id;
    }

    bool contains(const std::string& word) const {
        int node = kRoot;
        int length = word.size();
        for (int index = 0; index < length; ++index) {
            node = nodes_[node].children[word[index] - 'a'];
            if (node == kNoNode) { // не смогли пойти
                return false;
            }
        }
        return nodes_[node].is_terminal; // дошли, но в терминал ли?
    }

protected:
    struct Node {
        int children[kAlphabetSize]; // рёбра бора: kNoNode, если ребра нет
        bool is_terminal = false;
        int pattern_id = -1; // номер паттерна, который здесь заканчивается

        Node() {
            for (int letter = 0; letter < kAlphabetSize; ++letter) {
                children[letter] = kNoNode;
            }
        }
    };

    std::vector<Node> nodes_;
};
```

Вернёмся к задаче поиска многих паттернов. Бор уже даёт нам две вещи:

- все паттерны хранятся вместе, общие префиксы — один раз;
- за $O(|S|)$ можно проверить, является ли строка S одним из паттернов, — и это время не зависит от числа паттернов k .

Отсюда сразу получается первое решение задачи.

Алгоритм 2: Наивный поиск по бору

Для каждой позиции i текста идём из корня по буквам $T[i], T[i+1], \dots$, пока есть ребро. Каждая терминальная вершина на этом пути — вхождение паттерна, начинающееся в позиции i .

Время работы

Из каждой позиции мы проходим не больше $L = \max_j |P_j|$ рёбер, итого $O\left(\sum_{i=1}^k |P_i| + |T| \cdot L\right)$. Это уже не зависит от k : бор проверяет все паттерны *одновременно*, а не по очереди, как k запусков КМП.

Но если паттерны длинные, этого мало. Пусть паттерн один, $P = a^m$, а текст $T = a^n$: из каждой позиции мы честно проходим m рёбер, итого $O(nm)$. Причина та же, что у наивного поиска одной строки: сдвинувшись на следующую позицию, мы возвращаемся в корень и выбрасываем всё, что уже узнали о прочитанном тексте. В первой лекции это лечил КМП — префикс-функция говорила, какая часть прочитанного ещё пригодится. Хочется того же для бора: читать текст по одной букве и ни разу не возвращаться назад. Для этого бор надо превратить в *автомат* — этим и займёмся дальше.

1.2 Автоматы

НКА и ДКА подробно разбирались в курсе формальных языков, поэтому здесь — только определения и факты, которые понадобятся дальше.

Определение 1 (НКА). *Недетерминированным конечным автоматом (НКА) назовём кортеж $(Q, \Delta, \Sigma, q_0, F)$, где*

- Q — множество состояний, $|Q| < \infty$;
- $\Delta \subseteq Q \times \Sigma^* \times Q$ — множество переходов: из состояния по слову из Σ^* в состояние;
- Σ — алфавит;
- $q_0 \in Q$ — начальное состояние;
- $F \subseteq Q$ — множество конечных (принимающих) состояний.

Определение 2 (обывательское). *Автомат принимает слово, если он может пройти от q_0 до какого-нибудь $q_f \in F$, читая на рёбрах соответствующие куски слова. Два автомата эквивалентны, если множества принимаемых ими слов (распознаваемые языки) равны.*

Утверждение 1. *В произвольном НКА можно, сохранив распознаваемый язык:*

1. *сократить F до одного (быть может, фиктивного) состояния q_F ;*
2. *сделать все переходы не более чем однобуквенными;*
3. *избавиться от ε -переходов.*

Определение 3 (ДКА). *Детерминированным конечным автоматом (ДКА) назовём кортеж $(Q, \delta, \Sigma, q_0, F)$ с теми же Q, Σ, q_0, F , в котором переходы только однобуквенные и из каждого состо-*

яния по каждой букве может быть не более одного перехода. То есть $\delta : Q \times \Sigma \rightarrow Q$ — (частичная) функция. Если переход определён всегда, ДКА называют полным.

Проще говоря, ДКА читает слово по одной букве и в каждый момент точно знает, в каком он состоянии: никаких «может пройти», путь всегда один.

Утверждение 2. Произвольный НКА можно превратить в эквивалентный ДКА за $O(2^{|Q|} \cdot \text{poly}(|Q|))$.

Примечание. Зачем всё это в лекции про строки? Потому что Ахо-Корасик — это ДКА, построенный по набору паттернов:

- состояния Q — вершины бора;
- начальное состояние q_0 — корень;
- функция перехода δ — функция то, которую мы определим ниже (она определена для любой вершины и любой буквы, так что автомат ещё и полный);
- принимающие состояния F — вершины, у которых какой-то суффикс слова является паттерном.

Автомат читает текст по букве, и каждый раз, когда он оказывается в состоянии из F , в тексте только что закончилось вхождение какого-то паттерна. И, в отличие от общего случая, строится этот ДКА не за экспоненту, а за $O\left(|\Sigma| \sum_{i=1}^k |P_i|\right)$ — линейно от размера бора.

1.3 Алгоритм Ахо-Корасик

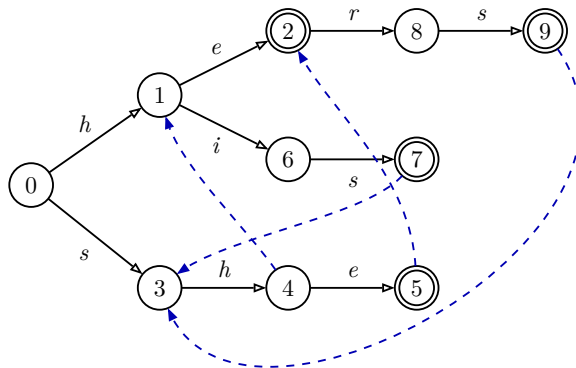
Вспомним, как работает КМП. Мы идём по тексту и храним k — длину самого длинного префикса паттерна, который совпадает с концом прочитанного текста. Пришла новая буква: если она продолжает паттерн — k растёт; если нет — мы откатываемся $k = \pi[k]$ и пробуем снова.

Ахо-Корасик делает ровно то же самое, но паттернов много. «Префикс паттерна» — это теперь вершина бора. Значит, мы храним вершину u бора такую, что $[u]$ — самый длинный конец прочитанного текста, который читается в боре. Остаётся понять, куда «откатываться», если из u нет ребра по новой букве. Ответ — по суффиксной ссылке, это аналог π .

Определение 1 (Суффиксная ссылка). Суффиксной ссылкой вершины u называют такую вершину $v = \text{link}(u)$, что $[v]$ является максимальным по длине собственным суффиксом $[u]$, который можно прочесть, идя по бору из корня (до любой вершины, даже не терминальной).

Для корня суффиксная ссылка не определена (у пустой строки нет собственных суффиксов). Доопределим её фиктивной вершиной NIL .

Пустая строка — суффикс любой строки и всегда читается в боре (это корень), поэтому у любой вершины, кроме корня, ссылка определена. В частности, у детей корня $\text{link} = \text{root}$.



На картинке — тот же бор с суффиксными ссылками: пунктиром нарисованы ссылки, не ведущие в корень (ссылки всех остальных вершин ведут в корень). Например, $\text{link}(5) = 2$: самый длинный собственный суффикс «she», читающийся в боре, — «he». А $\text{link}(9) = 3$: у «hers» суффиксы «ers» и «rs» в боре не читаются, а «s» — читается.

Замечание 1. Если в боре одна строка P , то вершины — это префиксы P , и $\text{link}(\text{vertex}(P.\text{substr}(0, i))) = \text{vertex}(P.\text{substr}(0, \pi[i]))$. То есть суффиксные ссылки — это ровно префикс-функция, просто «нарисованная на бору». Всё, что дальше, — обобщение КМП на много паттернов.

Лемма 1 (о цепочке ссылок). Для вершины u последовательность $u, \text{link}(u), \text{link}(\text{link}(u)), \dots, \text{root}$ — это вершины всех суффиксов $[u]$, которые читаются в боре, в порядке убывания длины.

Доказательство. По определению $\text{link}(u)$ — самый длинный собственный суффикс $[u]$ из бора, так что между $[u]$ и $[\text{link}(u)]$ по длине никаких суффиксов из бора нет. Пусть теперь s — суффикс $[u]$ из бора, который короче $[\text{link}(u)]$. Оба они — суффиксы одной строки $[u]$, значит более короткий s — суффикс более длинного $[\text{link}(u)]$. Тогда s — собственный суффикс $[\text{link}(u)]$ из бора, и по тому же рассуждению для вершины $\text{link}(u)$ следующий по длине такой суффикс — $\text{link}(\text{link}(u))$. Длина на каждом шаге строго уменьшается, поэтому цепочка конечна и заканчивается в корне.

Это то же самое, что в КМП: перебор $k, \pi[k], \pi[\pi[k]], \dots$ перебирает все супрефиксы по убыванию длины.

Определение 2 (Функция перехода). Введём функцию $\text{to}(u, c)$ — вершину, куда из вершины u можно перейти по букве c :

$$\text{to}(u, c) = \begin{cases} \text{vertex}([u] + c) & \text{если из } u \text{ есть ребро бора по } c \\ \text{to}(\text{link}(u), c) & \text{если ребра нет и } u \neq \text{root} \\ \text{root} & \text{если ребра нет и } u = \text{root} \end{cases}$$

Если по-простому, мы пытаемся дописать к $[u]$ букву c . Получилось — идём по ребру. Не получилось — отбрасываем от $[u]$ столько начала, сколько нужно, чтобы перейти к следующему по длине суффиксу из бора (это ровно $\text{link}(u)$), и пробуем ещё раз. А если дошли до пустой строки и дописать всё равно не можем — остаёмся в корне.

Пример 1. Посмотрим на картинку выше. Из вершины 5 («she») нет ребра по «r», поэтому $\text{to}(5, r) = \text{to}(\text{link}(5), r) = \text{to}(2, r) = 8$: из «he» ребро по «r» есть, и мы попадаем в «her». А из вершины 3 («s») нет ребра по «e», её ссылка ведёт в корень, а из корня ребра по «e» тоже нет — значит, $\text{to}(3, e) = \text{root}$.

Утверждение 1 (смысл функции перехода). $\text{to}(u, c)$ — это вершина, соответствующая самому длинному суффиксу строки $[u] + c$, который читается в боре.

Действительно, любой непустой суффикс $[u] + c$ имеет вид $s + c$, где s — суффикс $[u]$. Если $s + c$ читается в боре, то и s читается (бор замкнут относительно префиксов). Значит, нужно перебрать суффиксы s строки $[u]$ из бора по убыванию длины и найти первый, из вершины которого есть ребро по c . По лемме о цепочке эти суффиксы — это как раз $u, \text{link}(u), \text{link}(\text{link}(u)), \dots$, и функция to идёт ровно по этой цепочке. Если ни у кого ребра по c нет, самый длинный подходящий суффикс — пустой, то есть корень.

Утверждение 2 (ссылка ребёнка). Если из u есть ребро бора по букве c , то

$$\text{link}(\text{vertex}([u] + c)) = \text{to}(\text{link}(u), c)$$

(для $u = \text{root}$ считаем, что $\text{link}(\text{vertex}(c)) = \text{root}$).

Действительно, пусть $v = \text{vertex}([u] + c)$, $u \neq \text{root}$. Собственный суффикс $[v] = [u] + c$ — это либо пустая строка, либо $s + c$, где s — собственный суффикс $[u]$. Ищем самый длинный такой суффикс из бора — это ровно предыдущее рассуждение, только цепочку начинаем не с u , а с $\text{link}(u)$ (ведь s собственный). То есть ответ — $\text{to}(\text{link}(u), c)$.

Примечание. Тут очень помогает взгляд из матлога. Формулы для to и link выше — рекурсивные определения, и возникает вопрос: а корректны ли они, не заикнутся ли? Корректны, потому что каждый рекурсивный вызов уходит к вершине **строго меньшей глубины**:

- $\text{to}(u, c)$ ссылается на $\text{to}(\text{link}(u), c)$, а $\text{link}(u)$ мельче, чем u (собственный суффикс короче);
- $\text{link}(v)$ для v на глубине d с родителем u ссылается на $\text{link}(u)$ и $\text{to}(\text{link}(u), c)$ — вершины глубины меньше d .

Глубина — натуральное число, бесконечно убывать она не может, так что рекурсия всегда завершается. А раз всё выражается через «более мелкие» вершины, значения можно считать по индукции по глубине: база — корень (для него всё задано явно), переход — формулы выше. Обход вершин в порядке неубывания глубины — это и есть BFS.

Алгоритм 1: Ахо-Корасик — построение

1. Строим бор на данном наборе слов.
2. **База.** Для корня: $\text{to}(\text{root}, c) = \text{vertex}(c)$, если ребро по c есть, иначе root . Для детей корня $\text{link} = \text{root}$. Детей корня кладем в очередь.
3. **Переход.** Достаём из очереди вершину u . К этому моменту $\text{link}(u)$ уже посчитана (её посчитали, когда клали u в очередь), а у вершины $\text{link}(u)$ уже посчитаны все переходы $\text{to}(\text{link}(u), \cdot)$ (она мельче u , так что BFS её уже обработал). Для каждой буквы c :
 - если есть ребро бора в ребёнка v : $\text{to}(u, c) = v$, $\text{link}(v) = \text{to}(\text{link}(u), c)$, кладем v в очередь;
 - если ребра нет: $\text{to}(u, c) = \text{to}(\text{link}(u), c)$.

Почему это работает? Обе формулы взяты из утверждений выше и используют только вершины меньшей глубины, а для них по предположению индукции (BFS обрабатывает вершины в порядке неубывания глубины) всё уже посчитано корректно.

Время работы

Каждую вершину бора мы достаём из очереди один раз и тратим на неё $O(|\Sigma|)$ — по $O(1)$ на букву. Вершин не больше $\sum_{i=1}^k |P_i| + 1$, итого $O\left(|\Sigma| \sum_{i=1}^k |P_i|\right)$. Заметьте, что откатов по ссылкам в цикле, как в КМП, здесь нет вообще: откат «уже упакован» в готовую таблицу $\text{to}(\text{link}(u), \cdot)$.

Примечание. Вместо BFS можно считать to и link лениво — рекурсией с запоминанием, прямо по формулам из определений: $\text{Link}(v)$ вызывает $\text{Go}(\text{Link}(\text{parent}), c)$, а $\text{Go}(u, c)$ при отсутствии ребра вызывает $\text{Go}(\text{Link}(u), c)$. Корректность — ровно то же рассуждение про убывание глубины. Минус — глубокая рекурсия на длинных паттернах.

Реализация (C++): класс автомата Ахо-Корасик

Автомат — это тот же бор, к которому добавлены функция перехода `to` и две ссылки. Поэтому унаследуем его от `Trie`: вершины и `add_pattern` достаются даром, а в новом классе хранятся только `to_`, `link_` и `compressed_`. Автомат строится один раз, в конструкторе, по всему набору паттернов (добавлять паттерны после построения нельзя — ссылки пришлось бы пересчитывать).

```
class AhoCorasick : public Trie {
public:
    AhoCorasick(const std::vector<std::string>& patterns) {
        int count = patterns.size();
        for (int id = 0; id < count; ++id) {           // шаг 1
            add_pattern(patterns[id], id);
        }
        build_links();                                 // шаги 2–3
    }

    // occurrences — пары (номер паттерна, позиция конца вхождения в тексте)
    void find_occurrences(const std::string& text,
                          std::vector<std::pair<int, int>>& occurrences) const;

private:
    void build_links();

    std::vector<std::vector<int>> to_; // to_[u][c] — функция перехода to(u, c)
    std::vector<int> link_;           // суффиксные ссылки
    std::vector<int> compressed_;    // сжатые суффиксные ссылки (см. ниже)
};
```

1.4 Поиск множества паттернов

Определение 1 (Постановка задачи). Пусть есть набор паттернов $P_1, \dots, P_k$ и текст T , в котором мы хотим найти вхождения всех паттернов. Как это сделать быстрее, чем наивный КМП, который отработает за $O\left(k|T| + \sum_{i=1}^k |P_i|\right)$?

Будем читать текст автоматом: начинаем в корне и на каждую букву делаем переход по `to`. Главное здесь — понять, что именно помнит текущее состояние. Это говорит следующая теорема.

Теорема 1 (главный инвариант). Пусть $state_i$ — вершина, в которой мы оказались, прочитав $T[0..i]$. Тогда $[state_i]$ — самый длинный суффикс строки $T[0..i]$, который читается в боре.

Доказательство. Индукция по i . База: до чтения текста мы в корне, прочитана пустая строка, её самый длинный суффикс из бора — пустой. Переход: пусть утверждение верно для $X = T[0..i-1]$ и $u = state_{i-1}$, прочитали букву $c = T[i]$. Любой непустой суффикс $X + c$ из бора имеет вид $s + c$, где s — суффикс X из бора. По предположению индукции $|s| \leq |[u]|$, а значит s — суффикс $[u]$ (оба — суффиксы X). Получается, самый длинный суффикс $X + c$ из бора совпадает с самым длинным суффиксом $[u] + c$ из бора, а это по утверждению о смысле функции перехода и есть $to(u, c) = state_i$.

Следствие 1. Паттерн P_j заканчивается в позиции i текста $\Leftrightarrow$ вершина $vertex(P_j)$ лежит на цепочке суффиксных ссылок $state_i, link(state_i), \dots$

Действительно, P_j заканчивается в позиции $i \Leftrightarrow P_j$ — суффикс $T[0..i]$. Так как P_j читается в боре, по инварианту $|P_j| \leq |[state_i]|$, а так как оба — суффиксы $T[0..i]$, то P_j — суффикс $[state_i]$ из бора.

Реализация (C++): построение автомата

Построение ссылок и переходов — ровно шаги 2 и 3 алгоритма выше (заодно считаем и сжатые ссылки — они понадобятся при поиске, см. ниже):

```
void AhoCorasick::build_links() {
    int size = nodes_.size();
    to_.assign(size, std::vector<int>(kAlphabetSize, kRoot));
    link_.assign(size, kRoot);
    compressed_.assign(size, kRoot);

    std::queue<int> queue;
    for (int letter = 0; letter < kAlphabetSize; ++letter) { // шаг 2: база
        int child = nodes_[kRoot].children[letter];
        if (child != kNoNode) {
            to_[kRoot][letter] = child;
            link_[child] = kRoot;
            queue.push(child);
        } else {
            to_[kRoot][letter] = kRoot;
        }
    }
    while (!queue.empty()) { // шаг 3: переход
        int node = queue.front();
        queue.pop();
        int suffix = link_[node]; // мельче node — для неё всё уже посчитано
        if (suffix == kRoot || nodes_[suffix].is_terminal) {
            compressed_[node] = suffix;
        } else {
            compressed_[node] = compressed_[suffix];
        }
        for (int letter = 0; letter < kAlphabetSize; ++letter) {
            int child = nodes_[node].children[letter];
            if (child != kNoNode) { // ребро бора есть
                to_[node][letter] = child;
                link_[child] = to_[suffix][letter]; // ссылка ребёнка
                queue.push(child);
            } else { // ребра нет —
                to_[node][letter] = to_[suffix][letter]; // как из link(node)
            }
        }
    }
}
```

А по лемме о цепочке все такие суффиксы — это в точности вершины цепочки ссылок. Обратно, любая вершина цепочки — суффикс $[state_i]$, а значит, и $T[0..i]$.

Значит, находясь в $state_i$, надо пройти по цепочке ссылок и выписать все терминальные вершины. Но цепочка может быть длинной, а терминальных вершин на ней — мало. Поэтому заведём ссылки, которые перепрыгивают через нетерминальные вершины.

Определение 2 (Сжатая суффиксная ссылка). Сжатой суффиксной ссылкой вершины $u \neq \text{root}$ называют такую вершину $v = \text{comp}(u)$, что

$$\text{comp}(u) = \begin{cases} \text{link}(u) & \text{если } \text{link}(u) \text{ терминальная или } \text{link}(u) = \text{root} \\ \text{comp}(\text{link}(u)) & \text{иначе} \end{cases}$$

То есть это ближайшая терминальная вершина на цепочке ссылок, строго выше u (или корень, если терминальных там нет). Считается в том же BFS: $\text{link}(u)$ мельче u , так что $\text{comp}(\text{link}(u))$ уже известна.

Алгоритм 1: Ахо-Корасик — поиск

1. Построим автомат Ахо-Корасик на паттернах за $O\left(|\Sigma| \sum_{i=1}^k |P_i|\right)$.
2. Будем идти по функции to текстом T из корня: $\text{state} = \text{to}(\text{state}, T[i])$.
3. После каждого перехода прыгаем от state вверх по сжатым ссылкам, пока не придём в корень, и для каждой встреченной терминальной вершины выписываем вхождение. Важно: саму вершину state тоже надо проверить на терминальность — comp указывает *строго выше*.

Время работы

На первый взгляд кажется, что раз после каждой буквы мы прыгаем по цепочке ссылок, то на одну позицию может уйти много времени, и суммарно выйдет не $O(|T|)$. Но прыгаем мы по *сжатым* ссылкам, а каждый такой прыжок ведёт либо в корень (это последний прыжок, он один на позицию), либо в терминальную вершину — то есть даёт новое вхождение. Значит, всего прыжков $O(|T| + \text{ans})$, где ans — суммарное число вхождений всех паттернов в текст. Сами переходы по to — это $O(1)$ на букву, то есть ещё $O(|T|)$. Вместе с построением — $O\left(|\Sigma| \sum_{i=1}^k |P_i| + |T| + \text{ans}\right)$.

Быстрее ans не бывает в принципе — все вхождения надо хотя бы выписать.

Примечание. Почему нельзя прыгать по обычным ссылкам? Пусть паттерн один: $P = a^k$, текст $T = a^n$. Все вершины бора — $a, aa, \dots, a^k$, терминальная только последняя. Начиная с k -й буквы текста мы стоим в a^k , и цепочка обычных ссылок $a^k \rightarrow a^{k-1} \rightarrow \dots \rightarrow \text{root}$ имеет длину k — а вхождение на ней одно. Получаем $O(nk)$ прыжков на $n - k + 1$ вхождение. Сжатая ссылка из a^k ведёт сразу в корень.

Пример 1 (*he, she, his, hers* в тексте «ushers»). Номера вершин — как на картинке выше.

i	$T[i]$	state	цепочка по comp	вхождения
0	u	0 (ε)	—	—
1	s	3 (s)	$3 \rightarrow 0$	—
2	h	4 (sh)	$4 \rightarrow 0$	—
3	e	5 (she)	$5 \rightarrow 2 \rightarrow 0$	she, he
4	r	8 (her)	$8 \rightarrow 0$	—
5	s	9 (hers)	$9 \rightarrow 0$	hers

Самый интересный шаг — $i = 4$, это ровно пример с $\text{to}(5, r) = 8$ выше. Это «откат» из КМП: «she» + «r» в боре не читается, отбрасываем «s» и продолжаем с «he» + «r». А на $i = 3$ видно,

Реализация (C++): поиск паттернов

```
void AhoCorasick::find_occurrences(
    const std::string& text, std::vector<std::pair<int, int>>& occurrences) const {
    int state = kRoot;
    int length = text.size();
    for (int index = 0; index < length; ++index) {
        state = to_[state][text[index] - 'a'];           // шаг 2
        int node = state;
        while (node != kRoot) {                          // шаг 3
            if (nodes_[node].is_terminal) { // не терминальной может быть только state
                occurrences.push_back(std::make_pair(nodes_[node].pattern_id, index));
            }
            node = compressed_[node];
        }
    }
}
```

Метод `find_occurrences` объявлен в классе выше, здесь — его определение. Начало вхождения паттерна j , закончившегося в позиции `index`, — это `index - |Pj| + 1`.

зачем нужны сжатые ссылки: в одной позиции заканчиваются сразу два паттерна, и второй («he») мы находим только прыжком по `comp`.

Примечание. Если коротко, весь Ахо-Корасик держится на трёх фактах:

1. Цепочка суффиксных ссылок вершины u перебирает все суффиксы $[u]$ из бора по убыванию длины (как $k \rightarrow \pi[k]$ в КМП).
2. Состояние автомата после чтения куска текста — самый длинный конец этого куска, читающийся в боре.
3. Все `to` и `link` выражаются через вершины меньшей глубины, поэтому считаются по индукции — обходом в ширину.

Из 1 и 2 сразу следует, какие паттерны закончились в текущей позиции, а 3 говорит, как всё это быстро построить. Код при этом совсем короткий — сложность алгоритма не в реализации, а в том, чтобы понять, что хранит состояние.